

Orbit 포팅 매뉴얼

Orbit – RPi 제어 시스템 포팅 매뉴얼 (Deployment & Service Guide)

1. 개요 (Overview)

본 문서는 **Oribt (육아 홈 AIoT)** 프로젝트의 RPi 제어 시스템을 개발 환경 → 타겟 디바이스 (Raspberry Pi) 환경으로 배포하기 위한 **빌드 파이프라인 구성, 패키징, 서비스 데몬화 (systemd)** 절차를 정리한 포팅 매뉴얼이다.

본 매뉴얼을 따르면, RPi 전원 인가 시 제어 시스템이 자동 실행되며 STM32와 UART 통신, MQTT 기반 상태 송신, 센서 기반 주행 제어가 수행된다.

※ 본 매뉴얼은 현재 소스 코드에 MQTT 서버 정보, UART 디바이스 경로 등이 하드코딩된 상태를 기준으로 작성되었다.

※ 향후 Config 파일 분리 버전(v2)에서 개선될 예정이다.

2. 타겟 환경 (Target Environment)

- Device: Raspberry Pi 5
- OS: Ubuntu 24.04 (aarch64)
- Compiler: g++ (C++17)
- Dependencies:
 - mosquitto
 - libmosquitto-dev
 - libjsoncpp-dev

3. 디렉토리 구조 (Project Layout)

```
rpi/
├── comm/           # STM UART 통신 모듈
├── common/         # protocol, types 정의
├── control/        # drive_state, emergency_release
├── planner/        # speed/steer planner
└── sensor/        # MQTT 센서 수신
```

```
|— status/                # MQTT 상태 송신
|— scripts/
|   |— build.sh           # Release 빌드 스크립트
|   └— clean.sh           # 빌드 클린 스크립트
|— CMakeLists.txt
└— main.cpp
```

4. 의존성 설치

```
sudo apt update
sudo apt install -y \
    build-essential \
    cmake \
    libmosquitto-dev \
    libjsoncpp-dev
```

5. 빌드 파이프라인 정립 (Build Pipeline)

5.1 빌드 스크립트

```
# scripts/build.sh
#!/bin/bash
set -e

ROOT_DIR=$(cd "$(dirname "$0")/.." && pwd)
BUILD_DIR="$ROOT_DIR/build"

echo "Project: $ROOT_DIR"
echo "Build dir: $BUILD_DIR"

rm -rf "$BUILD_DIR"
mkdir -p "$BUILD_DIR"
cd "$BUILD_DIR"

cmake -DCMAKE_BUILD_TYPE=Release ..
make -j$(nproc)
```

```
echo "Build done."  
echo "> Run: $BUILD_DIR/rpi_controller"
```

5.2 빌드 수행

```
chmod +x scripts/build.sh  
./scripts/build.sh
```

5.3 실행 파일 생성

```
build/rpi_controller
```

6. 패키징 & 배포 구조 (Packaging & Install Layout)

6.1 배포 디렉토리 설치 경로

```
sudo mkdir -p /opt/yua-care/bin  
sudo mkdir -p /opt/yua-care/logs
```

6.2 바이너리 배포

```
sudo cp build/rpi_controller /opt/yua-care/bin/
```

6.3 런타임 실행 스크립트(run.sh)

```
sudo nano /opt/yua-care/bin/run.sh
```

```
#!/bin/bash  
set -x    # 디버그 로그 활성화  
  
LOG_DIR=/opt/yua-care/logs  
APP=/opt/yua-care/bin/rpi_controller  
  
echo "[RUN] Starting YUA controller..."
```

```
echo "[RUN] LOG_DIR=$LOG_DIR"
echo "[RUN] APP=$APP"

mkdir -p "$LOG_DIR"
touch "$LOG_DIR/rpi_controller.log"

exec $APP >> "$LOG_DIR/rpi_controller.log" 2>&1
```

```
sudo chmod +x /opt/yua-care/bin/run.sh
```

7. 서비스 데몬화 (systemd)

7.1 서비스 파일 생성

```
sudo nano /etc/systemd/system/yua-care.service
```

```
[Unit]
Description=YUA Care Robot Controller
After=network.target

[Service]
ExecStart=/opt/yua-care/bin/run.sh
Restart=always
RestartSec=2
User=pi
WorkingDirectory=/opt/yua-care/bin

[Install]
WantedBy=multi-user.target
```

7.2 서비스 등록 및 실행

```
sudo systemctl daemon-reload
sudo systemctl enable yua-care
sudo systemctl start yua-care
```

7.3 상태 확인

```
systemctl status yua-care  
journalctl -u yua-care -f
```

8. 부팅 시 자동 실행 플로우

```
RPi 전원 인가  
↓  
systemd 부팅  
↓  
yua-care.service 실행  
↓  
/opt/yua-care/bin/run.sh 실행  
↓  
rpi_controller 실행  
↓  
STM UART 연결 & 제어 시작
```

배포 검증 단계 트러블 슈팅 정리 (Troubleshooting Log)

이슈 1. chmod 실행 안됨 (Windows PowerShell)

증상 :

```
'chmod' is not recognized...
```

원인 :

- Windows PowerShell에서는 chmod 미지원

해결 :

```
WSL(Ubuntu) 환경에서 실행  
chmod +x scripts/build.sh
```

이슈 2. CMake Error: Could NOT find PkgConfig

증상 :

```
Could NOT find PkgConfig (missing: PKG_CONFIG_EXECUTABLE)
```

해결 :

```
sudo apt update  
sudo apt install -y pkg-config
```

⚠ 이슈 3. systemd 서비스 실행 즉시 종료 (exit 255)

증상 :

```
STM UART not opened  
UART open failed: No such file or directory
```

원인

- WSL 환경에는 `/dev/ttyAMA2` , UART 통신 없음

해결

- 실제 RPi에서 테스트
- 또는 Dry-run 모드 처리:

```
if (!stm.isOpen()) {  
    std::cerr << "[WARN] STM UART not found. Running in DRY-  
    RUN mode.\n";  
}
```

⚠ 이슈 4. Permission denied (log 파일)

증상 :

```
/opt/yua-care/logs/rpi_controller.log: Permission denied
```

해결 :

```
sudo chown -R root:root /opt/yua-care  
sudo chmod -R 755 /opt/yua-care
```